Lecture 3: Introduction to Software Process Models

Vedaj Padman [*jp_vedaj @cb.amrita.edu*]

3.1 Understanding Software

Software has become one of the most influential technologies in human history. Modern society depends on software not only for computation, but for communication, commerce, healthcare, transportation, governance, and entertainment. Despite this dependence, software is often misunderstood as merely a collection of programs written in a programming language. In reality, software is a far richer and more complex artifact.

Software can be understood as a combination of three essential elements. The first element is a set of instructions that, when executed, provide desired functionality and performance. The second element consists of data structures that enable the software to store, manipulate, and retrieve information efficiently. The third element is documentation, which describes the operation, use, and maintenance of the system. Without documentation, even correctly functioning software becomes difficult to evolve and maintain.

Unlike physical products, software is intangible. It cannot be touched, weighed, or physically inspected. Its value lies entirely in the behavior it exhibits when executed and the services it provides to its users. This intangibility makes software both powerful and dangerous: powerful because it can be easily replicated and distributed, and dangerous because defects are often invisible until the software is executed under specific conditions.

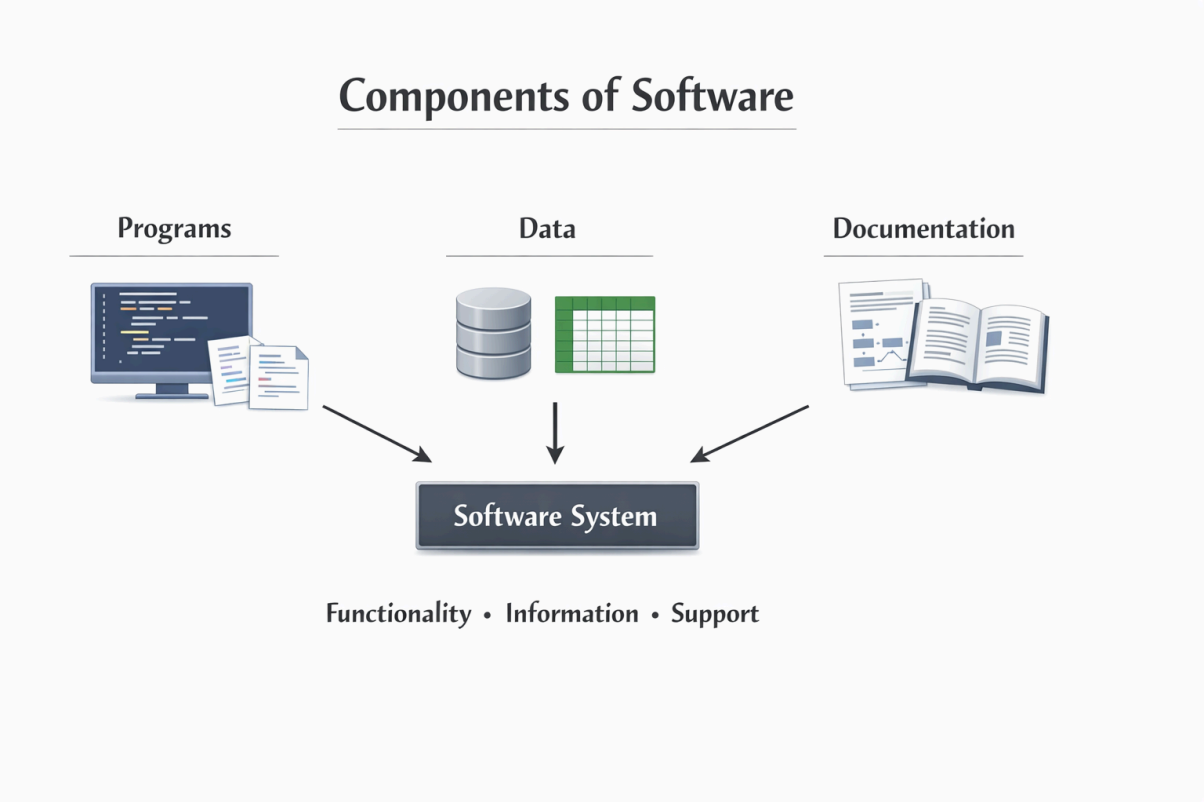


Figure 1: Components of Software – Programs, Data, and Documentation

3.2 The Unique Nature of Software

Software differs fundamentally from hardware in how it is created and how it fails. Hardware systems are manufactured through physical processes, and once produced, defects tend to arise due to wear and tear. Software, on the other hand, is developed through intellectual effort. It does not wear out in the physical sense. Instead, software deteriorates as it is modified to accommodate new requirements, platforms, or environments.

Each change made to a software system has the potential to introduce unintended side effects. Over time, these accumulated changes increase complexity, making the system harder to understand and maintain. This phenomenon explains why older software systems, even when still useful, are often described as legacy systems that are costly and risky to modify.

Another defining characteristic of software is its high degree of customization. While hardware industries have long relied on standardized components, most software systems are still built or heavily adapted for specific organizational needs. Although component-based development and reuse have improved, software engineering remains largely a discipline of controlled creativity rather than mass production.

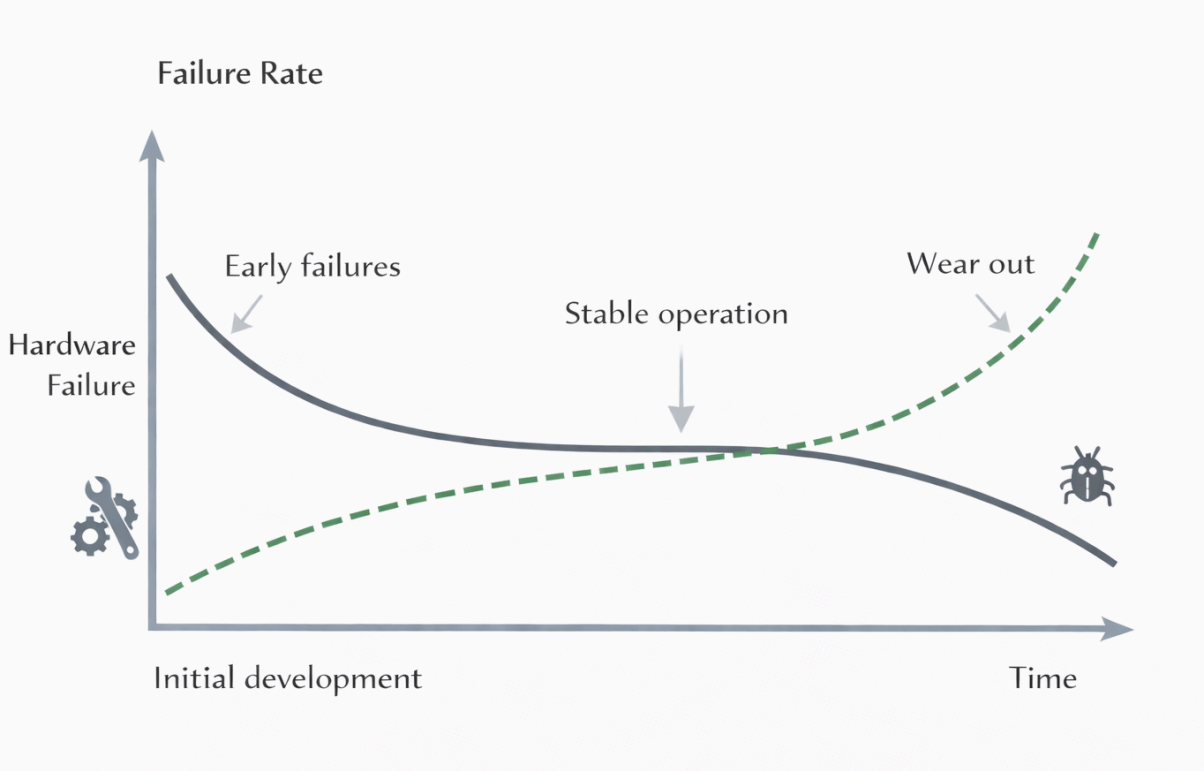


Figure 2: Hardware vs Software failure curves

3.3 The Software Crisis

During the 1960s and 1970s, as software systems grew in size and ambition, organizations began to experience widespread project failures. Large systems were delivered years late, exceeded their budgets by large margins, or failed to meet user expectations altogether. In some cases, failures had severe consequences, including financial loss and threats to human safety.

This period became known as the software crisis. Importantly, the crisis was not caused by a lack of programming ability. Many of the failed projects were staffed by highly skilled programmers. The

real problem lay in the absence of disciplined methods for managing complexity, coordinating teams, handling change, and ensuring quality.

The recognition that software development required more than ad-hoc coding led to the emergence of software engineering as a formal discipline. The goal was to bring the rigor, predictability, and accountability of traditional engineering fields into the development of software systems.

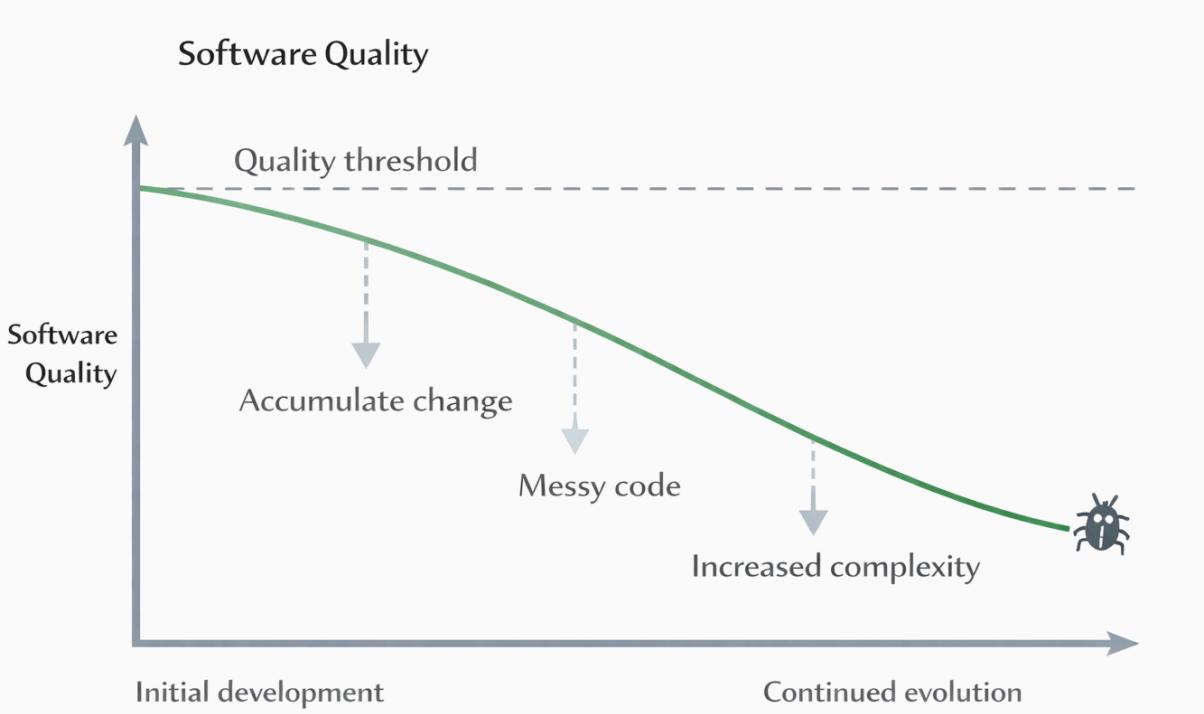


Figure 3: Software deterioration due to accumulated change

3.4 Defining Software Engineering

Software engineering can be defined as the application of systematic, disciplined, and quantifiable approaches to the development, operation, and maintenance of software. This definition emphasizes that software engineering is not limited to writing code. Instead, it encompasses the entire lifecycle of a software system, from initial concept to eventual retirement.

A software engineering approach seeks to answer several critical questions. How can we ensure that the software satisfies real user needs? How can we deliver systems on time and within budget? How can we build software that is reliable, maintainable, and adaptable to change? These questions highlight that software engineering is as much about decision-making and trade-offs as it is about technical implementation.

3.5 The Software Process

At the heart of software engineering lies the concept of process. A software process is a structured set of activities that leads to the creation of a software product. Regardless of the specific methodology used, every software process includes activities related to understanding requirements, designing solutions, implementing functionality, verifying correctness, and deploying the system.

It is important to note that a process is not a rigid recipe. Instead, it provides a framework within which teams can plan, organize, and control their work. Effective processes are adaptable, allowing teams to tailor activities and artifacts based on project size, complexity, risk, and organizational culture.

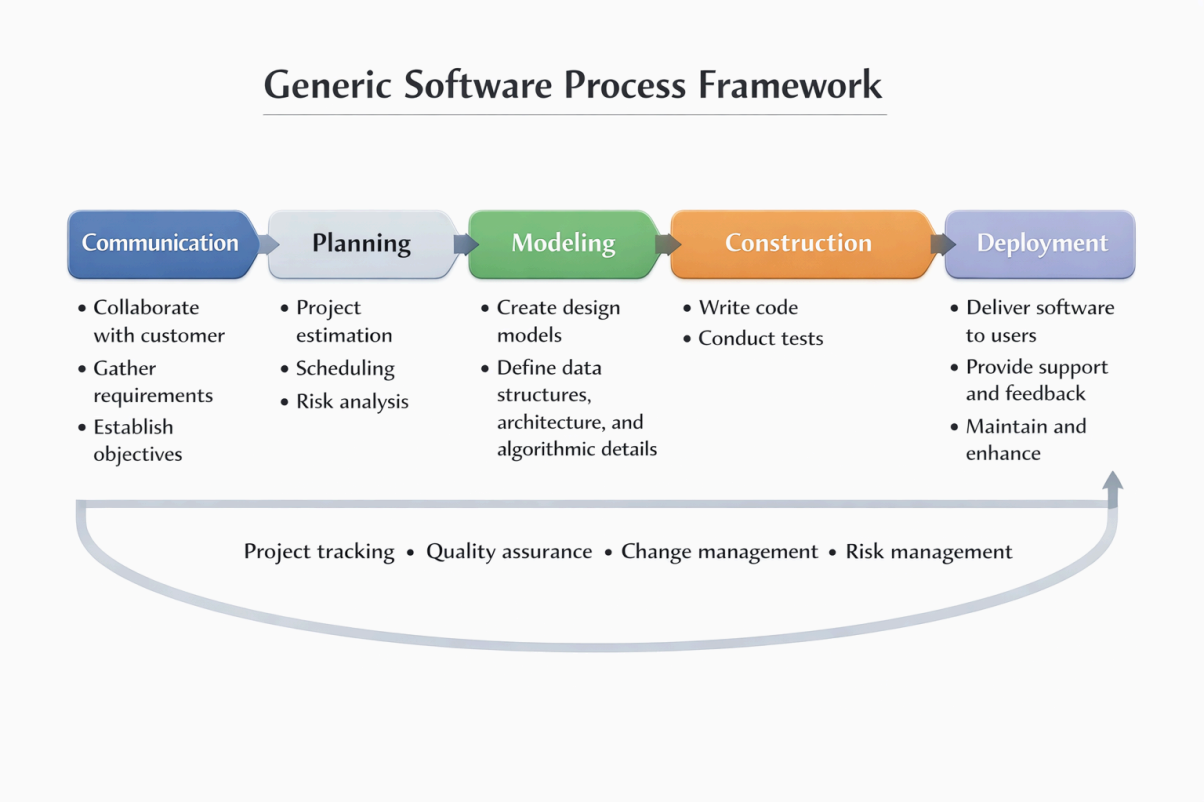


Figure 4: Generic Software Process Framework underlying all process models

3.6 Why Software Projects Fail

Despite decades of research and experience, software projects continue to fail at an alarming rate. Studies consistently show that technical issues are rarely the primary cause of failure. Instead, failures often stem from unclear requirements, unrealistic schedules, poor communication, lack of stakeholder involvement, and inadequate attention to quality.

One of the most significant challenges in software development is managing change. User needs evolve, technologies advance, and business environments shift. Processes that assume stable and complete requirements from the outset are often unable to cope with this reality. As a result, many projects either resist necessary change or collapse under its weight.

3.7 The Need for Adaptive Approaches

Traditional software process models emphasized extensive upfront planning and sequential execution of phases. While such approaches can be effective in stable and well-understood domains, they struggle in environments characterized by uncertainty and rapid change.

This limitation led to the emergence of adaptive, change-driven approaches to software development. These approaches recognize that learning occurs throughout the development process and that early assumptions are often incorrect. Instead of attempting to eliminate change, adaptive methods seek to manage it effectively through short feedback cycles and continuous stakeholder involvement.

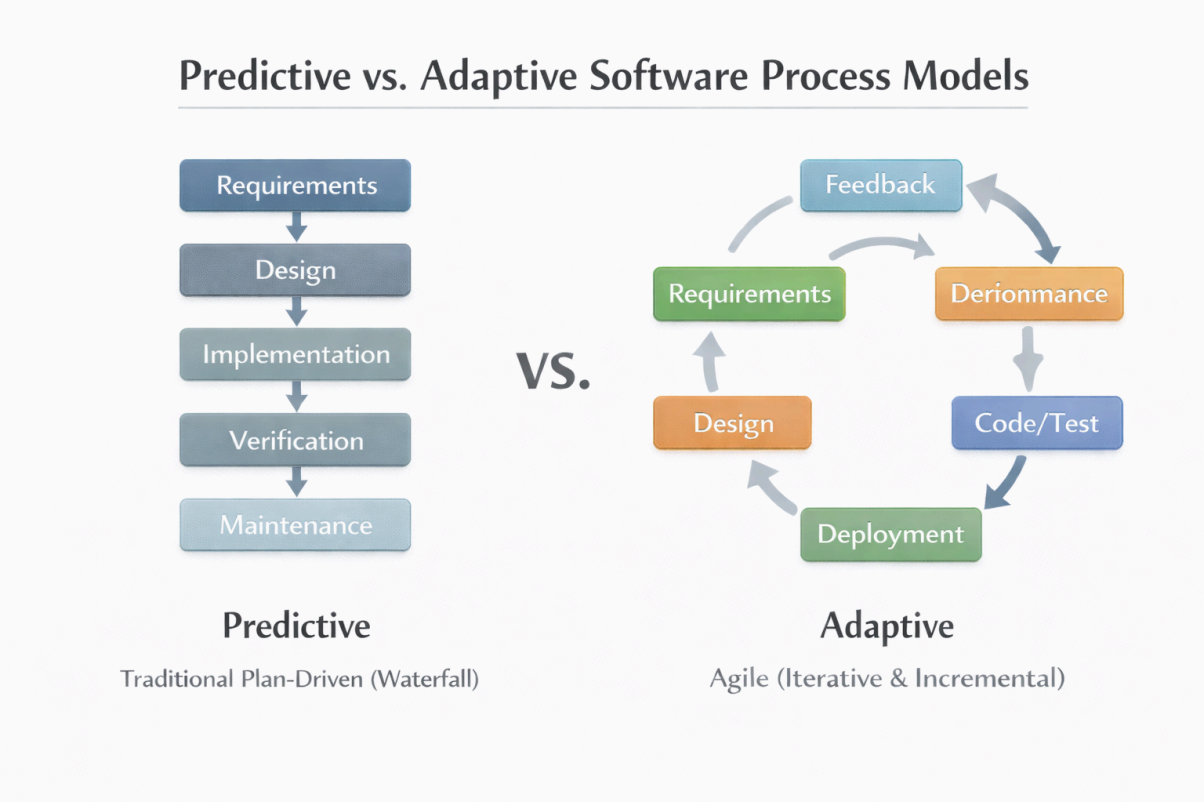


Figure 5: Predictive vs Adaptive software process models

3.8 Introduction to Agile Thinking

Agile software development represents a shift in mindset rather than a single prescribed methodology. Agile approaches emphasize collaboration, customer feedback, incremental delivery, and continuous improvement. The Agile Manifesto formalized these ideas by prioritizing individuals and interactions, working software, customer collaboration, and responsiveness to change.

It is essential to understand that Agile does not reject planning, documentation, or discipline. Rather, it challenges teams to focus on what creates the most value at a given moment. Agile practices are tools that support this philosophy, but adopting the tools without embracing the underlying mindset often leads to superficial and ineffective implementations.

3.9 Conclusion

Software engineering exists because building software is inherently complex, human-centric, and prone to uncertainty. Effective software development requires not only technical skill but also disciplined processes, thoughtful design, continuous learning, and collaboration among diverse stakeholders. The foundations established in this lecture provide the conceptual grounding necessary to understand both traditional and modern approaches to software process models.

Key Takeaways

- Software is an engineered artifact composed of programs, data, and documentation.
- Software: Programs, data, and documentation that together provide functionality.
- Software does not wear out physically; it deteriorates due to accumulated change.
- The software crisis revealed the limits of ad-hoc programming practices.
- Software engineering introduces disciplined, lifecycle-wide development practices.
- Every software system is built using a defined or implicit process.
- Most project failures are rooted in organizational and process issues.
- Adaptive approaches address uncertainty and evolving requirements.
- Agile development emphasizes collaboration, feedback, and learning.

Key Terminology

- **Software Engineering:** Systematic and disciplined approaches to software development and maintenance.
- **Software Process:** A structured framework of activities used to build and evolve software systems.
- **Software Crisis:** Widespread failure of early large-scale software projects.
- **Process Model:** An abstract representation of how software development activities are organized.
- **Traditional Process Models:** Plan-driven models emphasizing upfront planning and sequential phases.
- **Agile Development:** Adaptive development emphasizing collaboration and responsiveness to change.
- **Agile Manifesto:** A statement defining the core values and principles of Agile development.